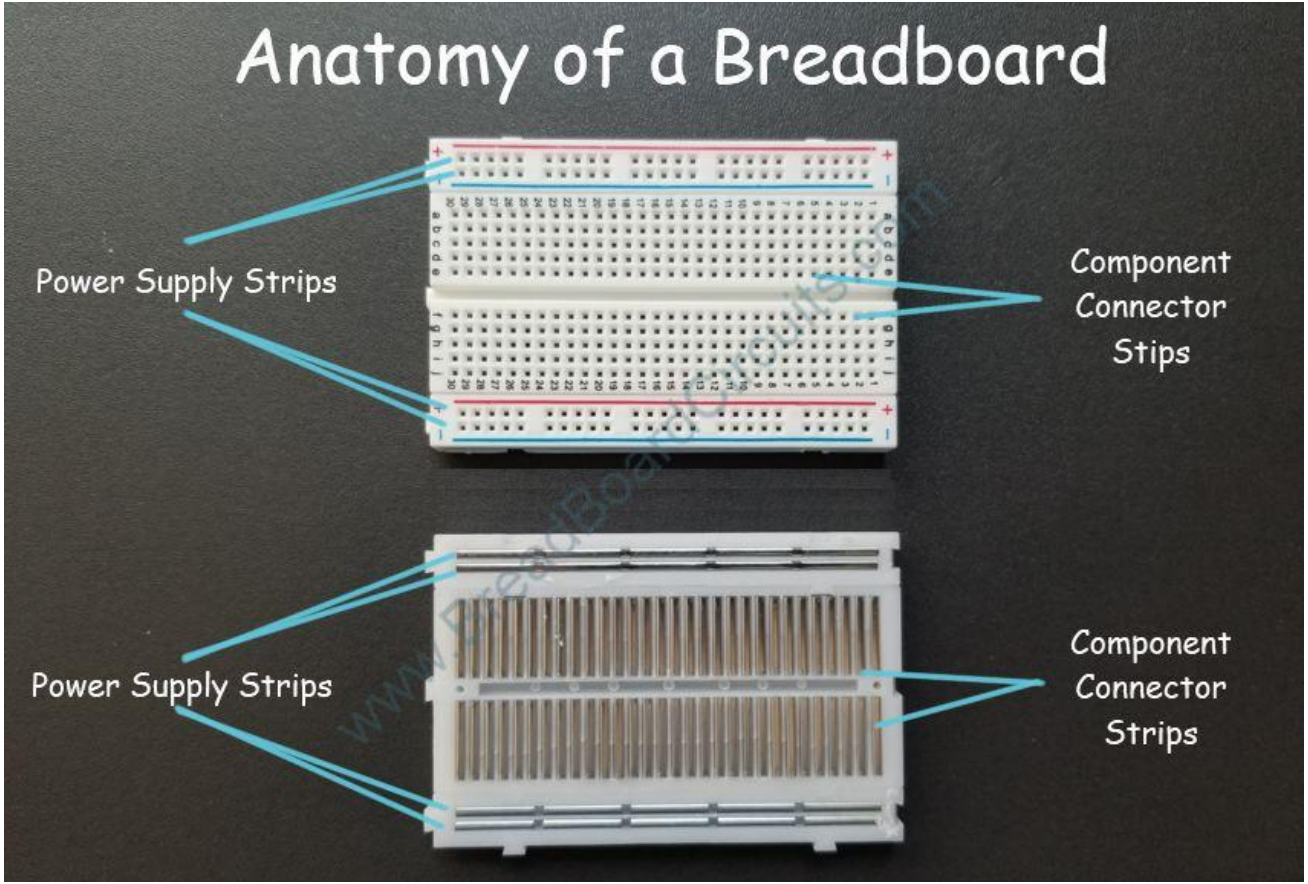
Session 1

Working with ESP32

Session 1 Topics

- Quick recap + Intro to electronics
- Working with ESP32, coding + upload
- Input components
  - Dip switch & micro switch
  - Push Button
  - Rotary encoder
  - Joystick
- Making simple sounds with the buzzer

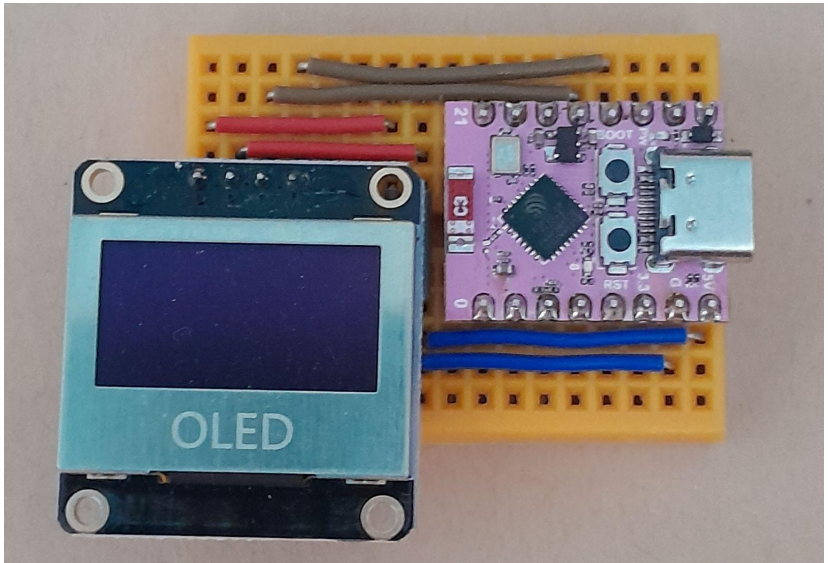
Intro to electronics

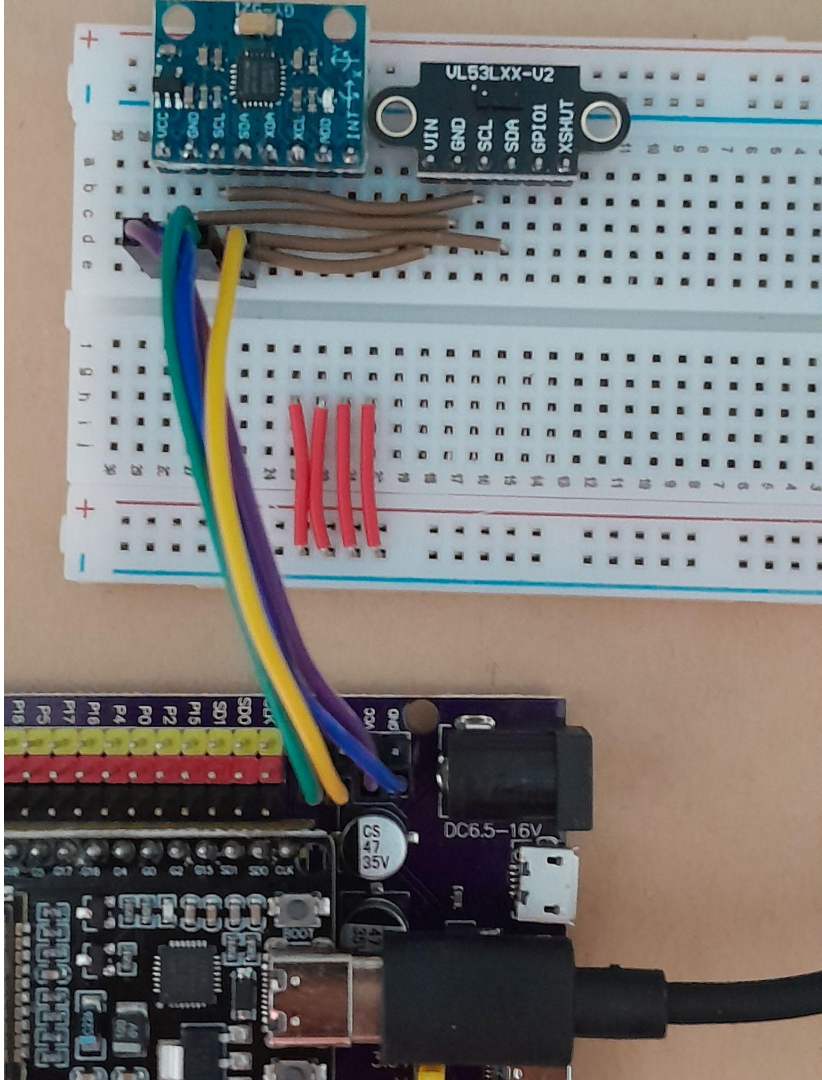


Src: https://www.breadboardcircuits.com

Intro to electronics

Jumper wires are used on the breadboard to connect different components to each other.



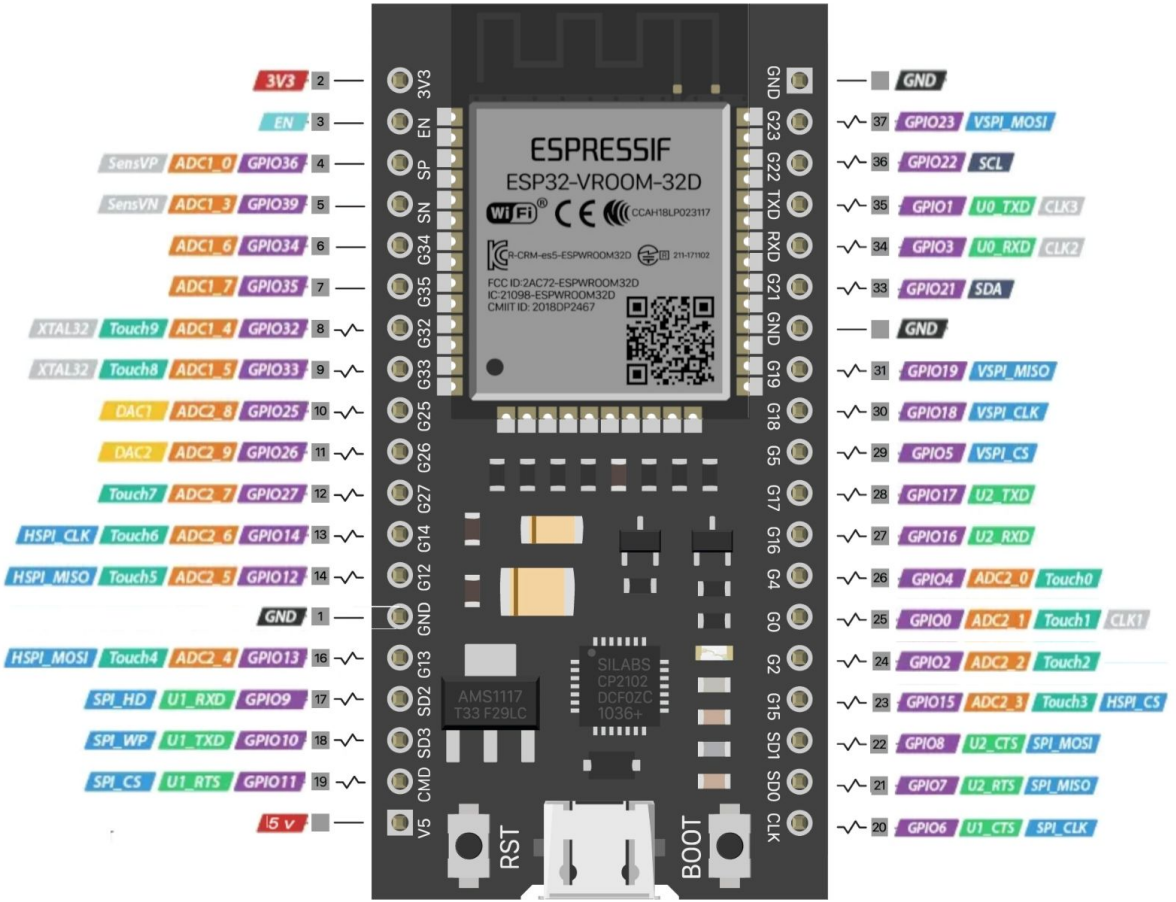


Working with ESP32

Each pin has multiple functions

Match the sensor or device you want to connect with the corresponding pin

E.g, a servo (type of motor) requires a PWM-enabled pin.



Working with ESP32

Important terms:

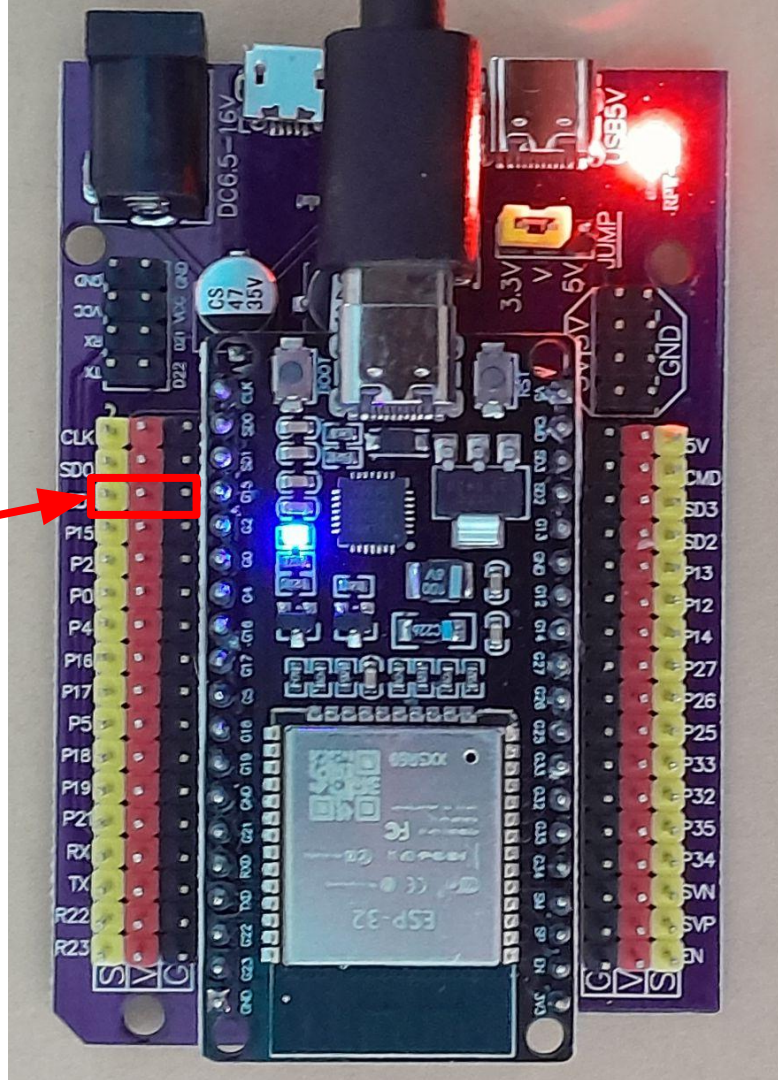
VCC = Power = either 5V or 3.3V = Positive(+) end of power source

GND = Ground = 0V = Negative(-) end of power source

Working with ESP32

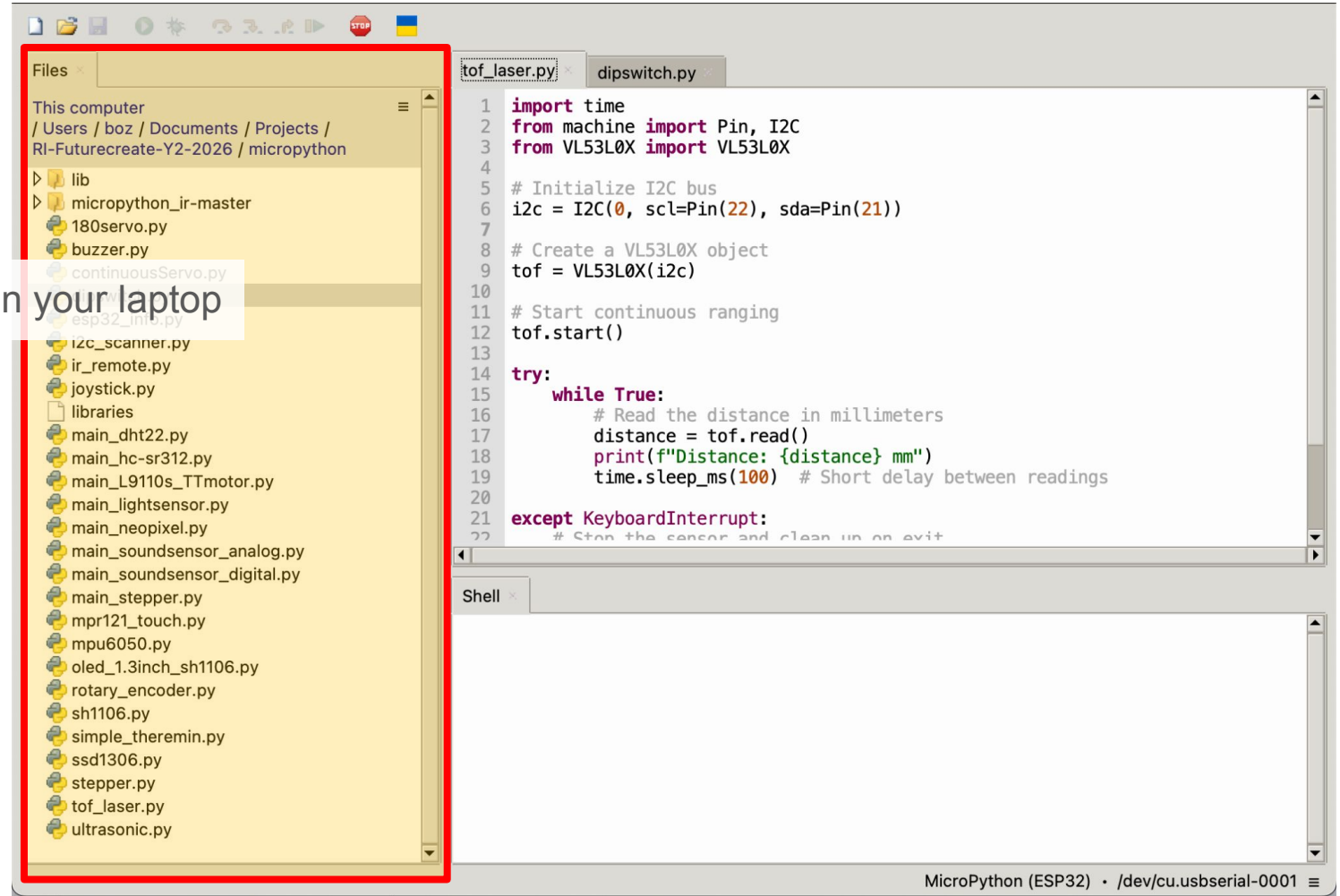
Esp32 breakout board

- Presents GPIO pins as Signal(yellow), Power(red) and Ground(Black)
- Most sensors and devices can be connected in this 3-pin configuration

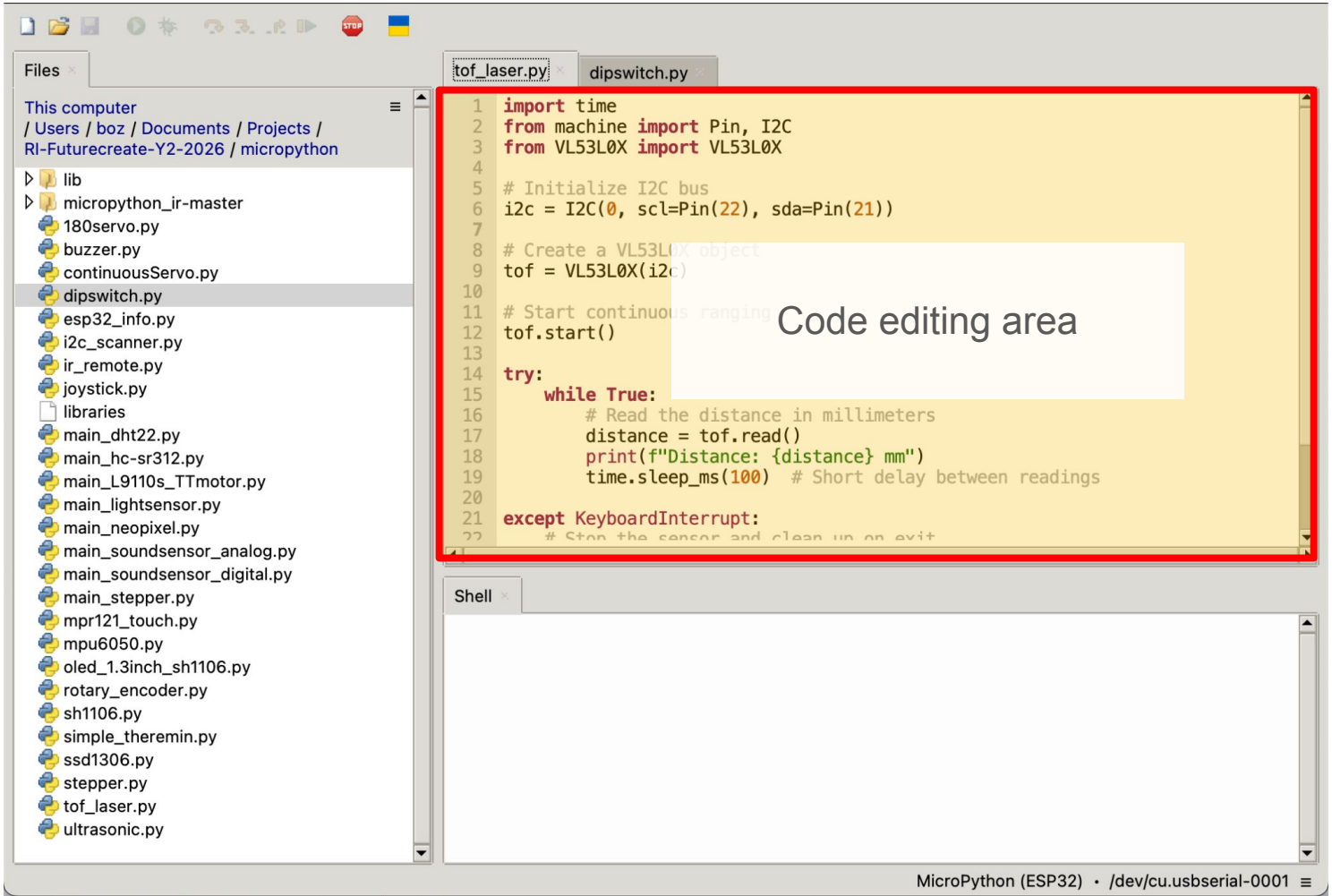


Intro to Thonny IDE

Files on your laptop

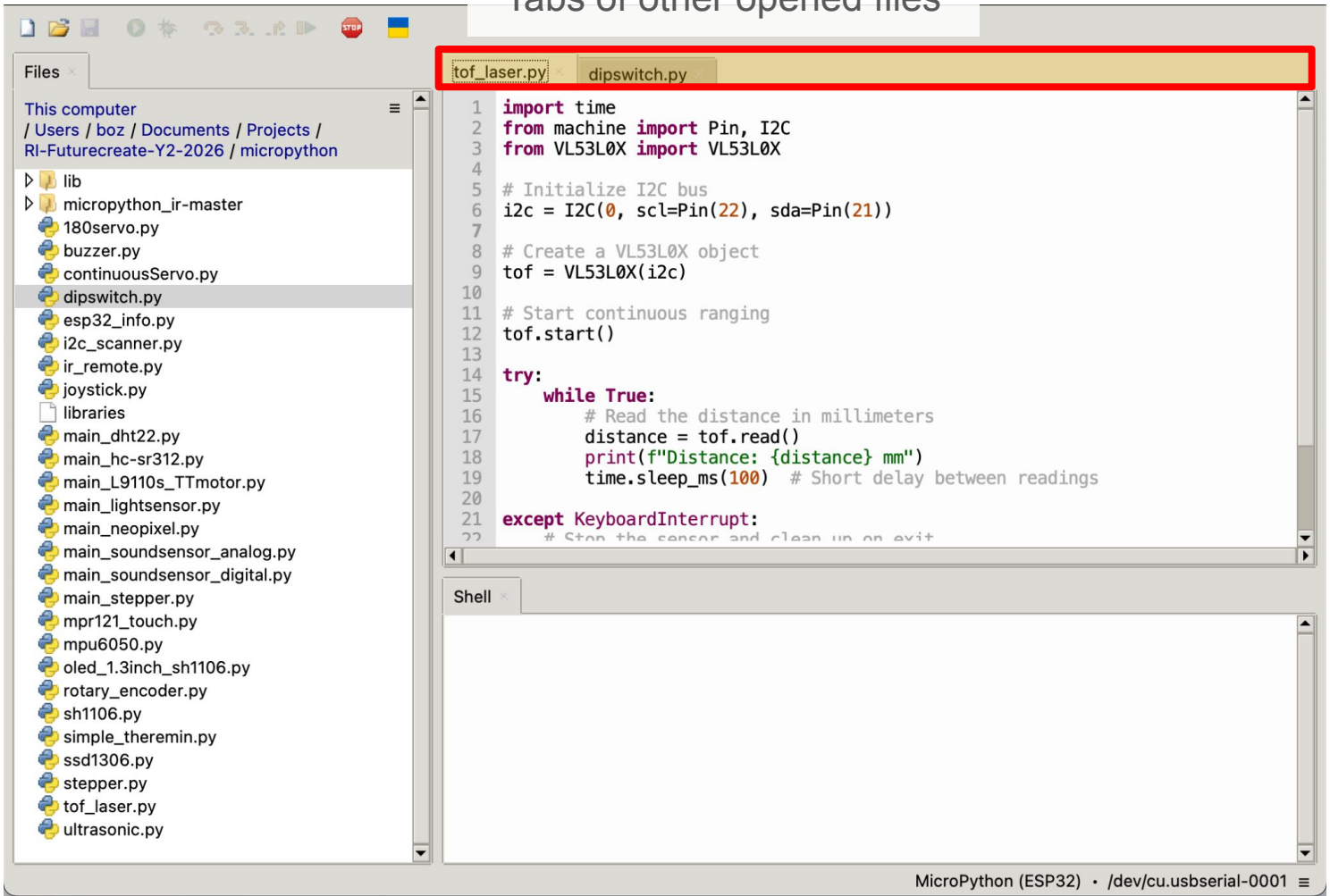


Intro to Thonny IDE



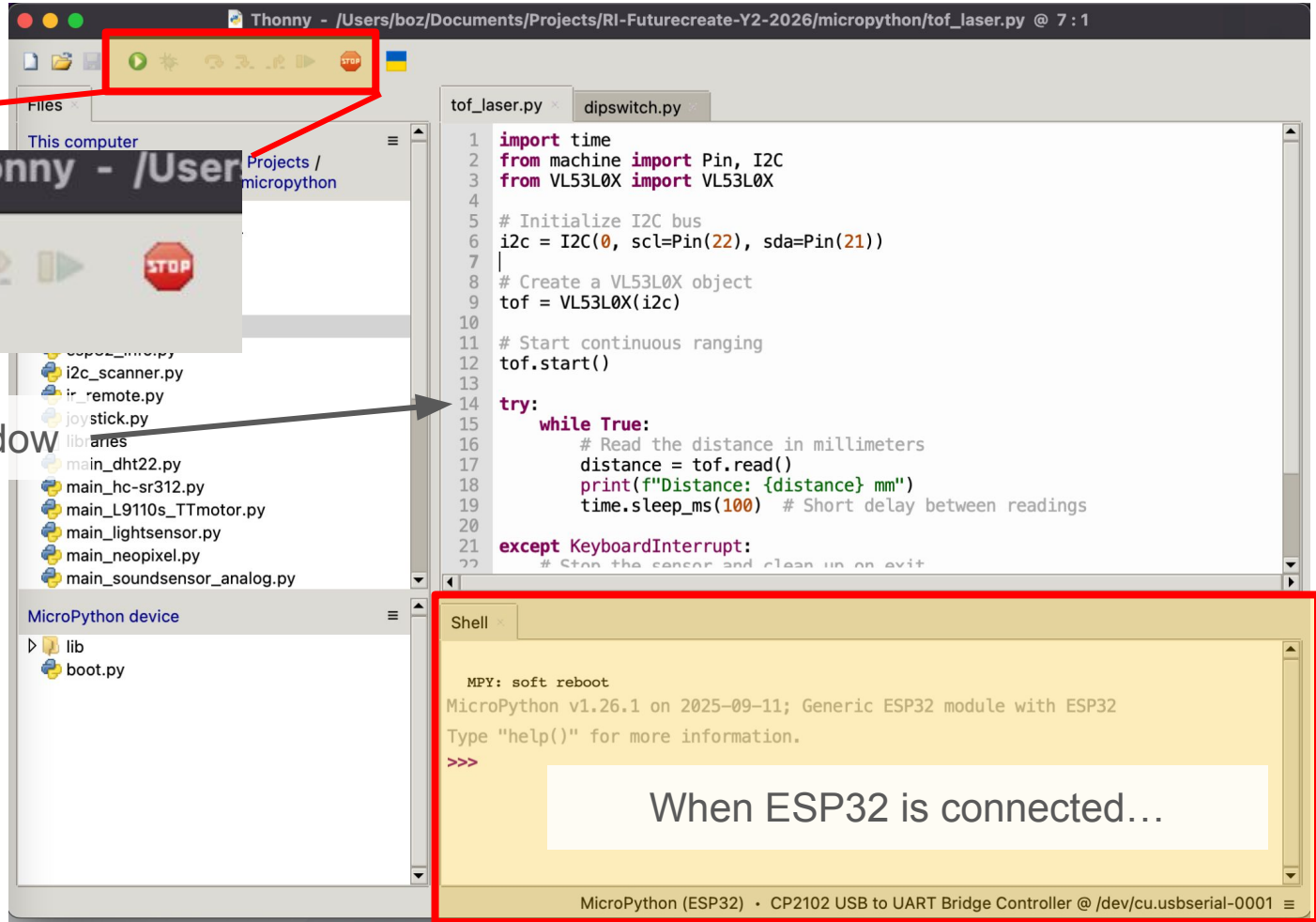
Intro to Thonny IDE

Tabs of other opened files

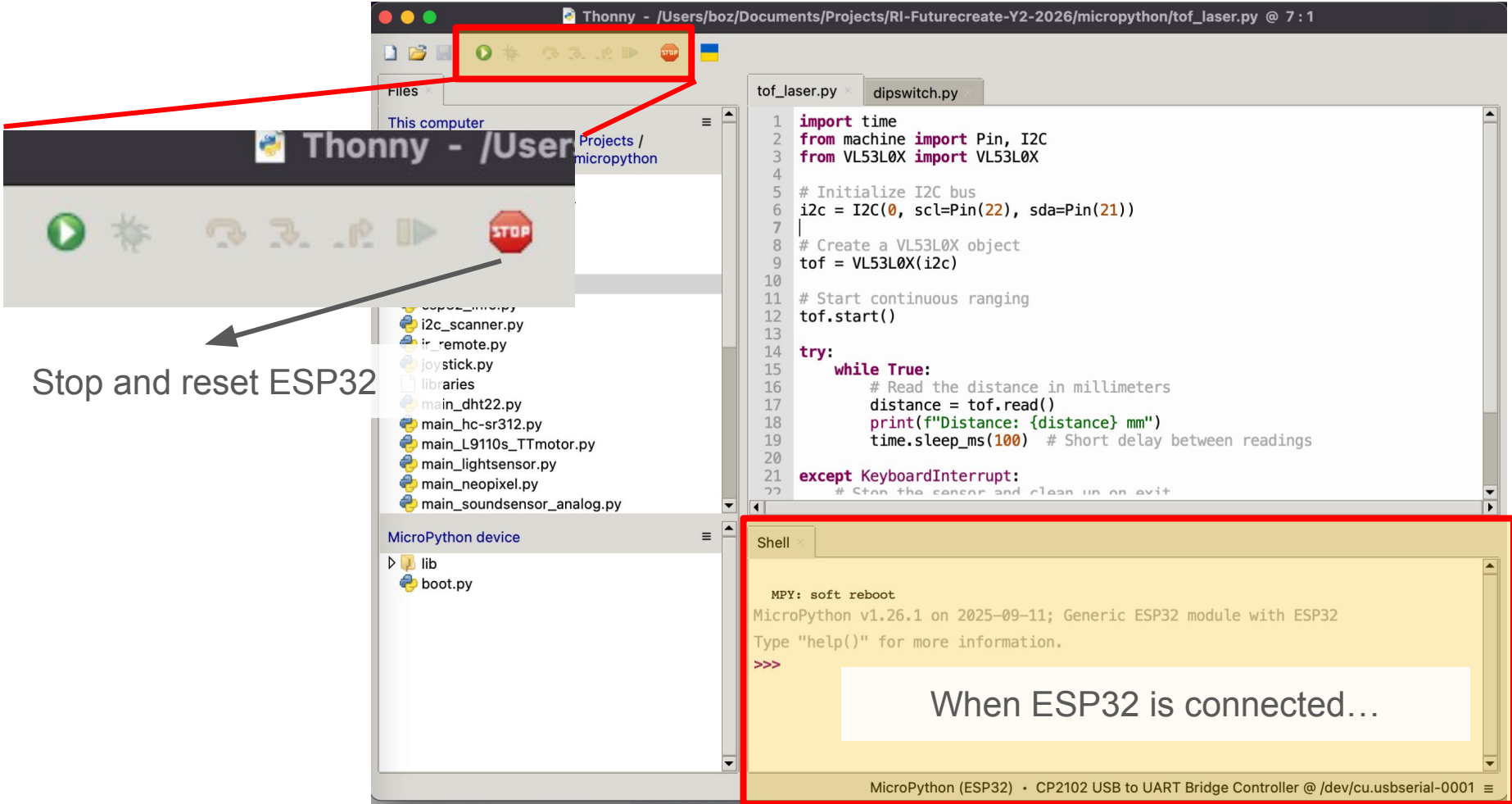


Intro to Thonny IDE

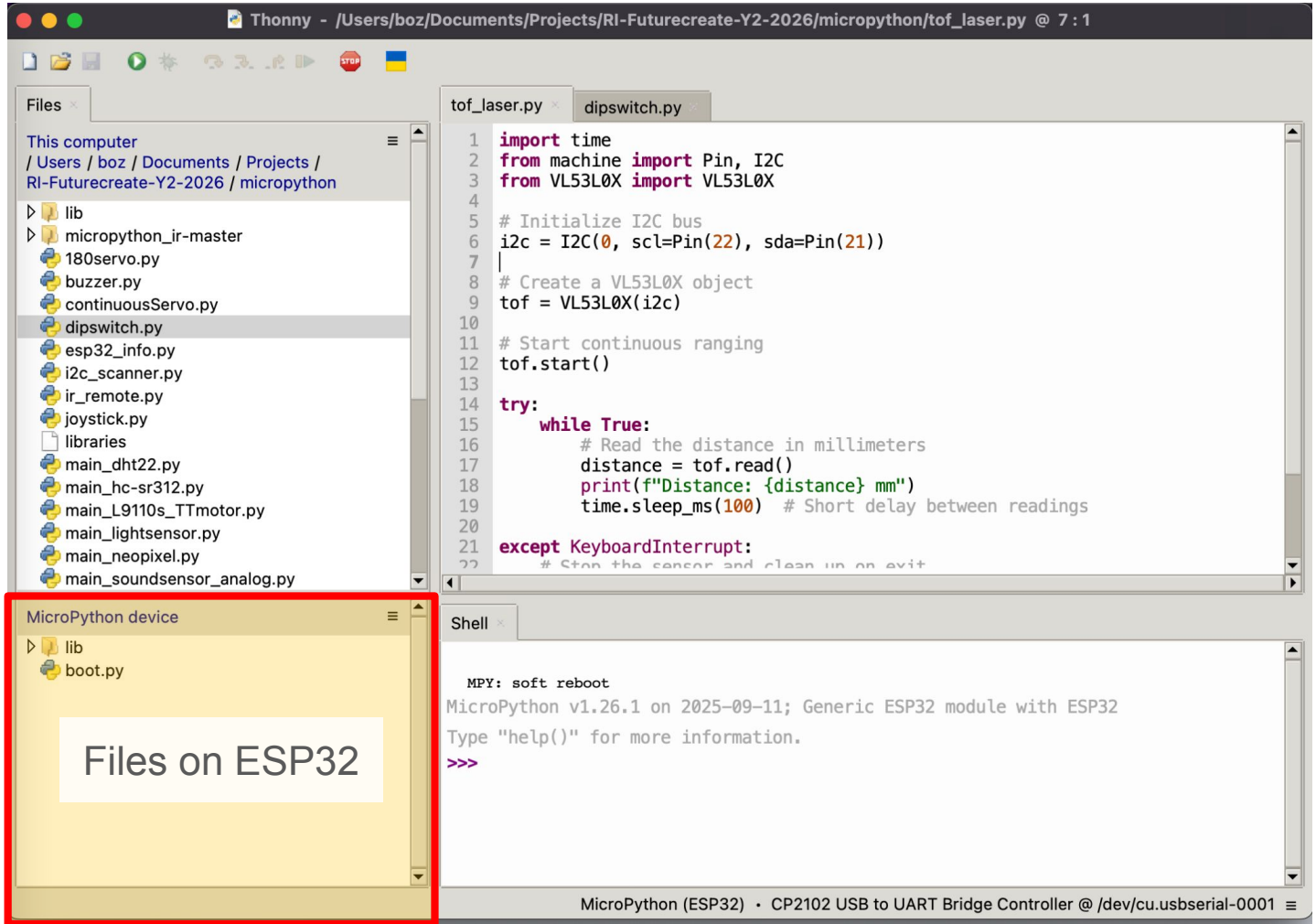
Run script in active window



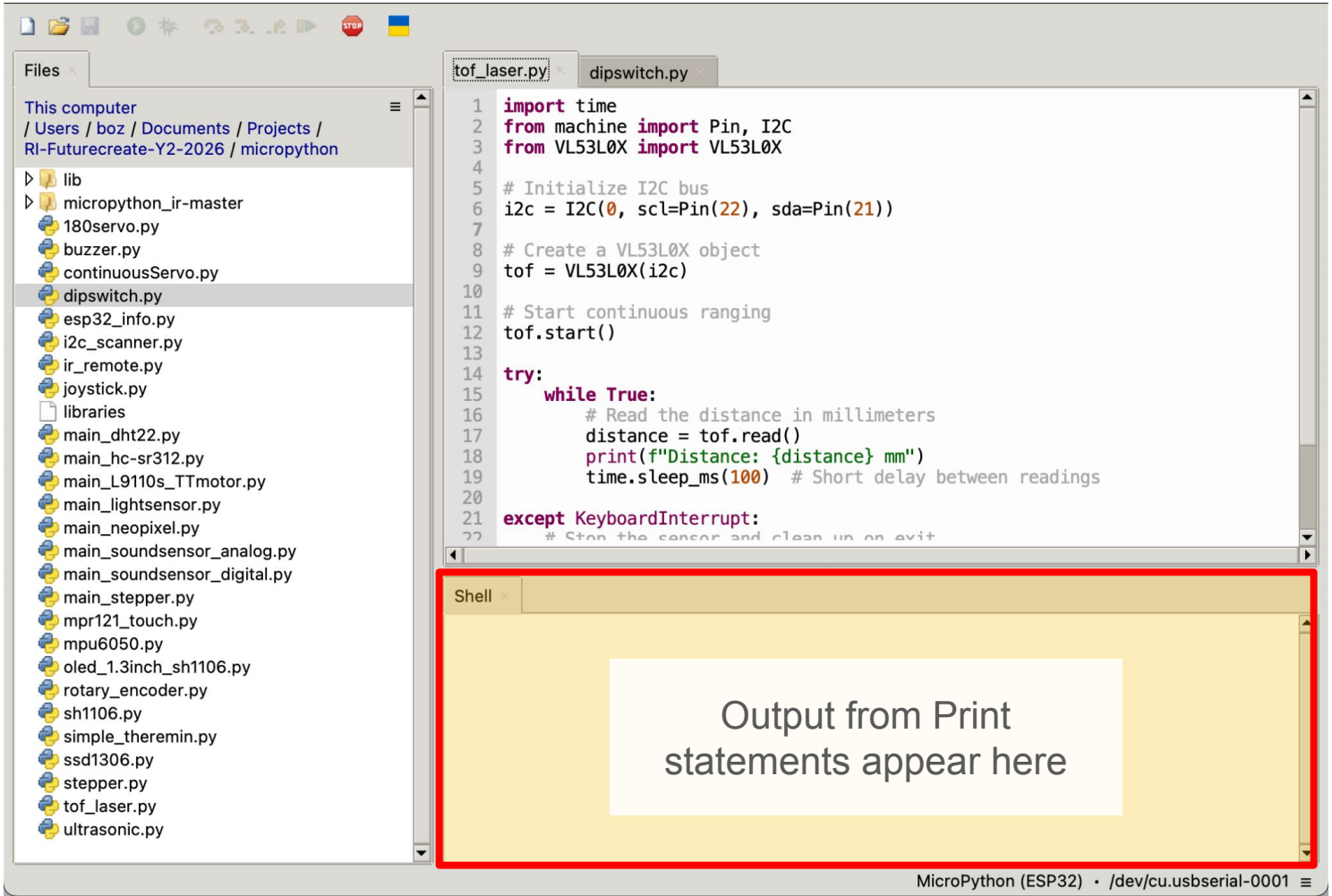
Intro to Thonny IDE



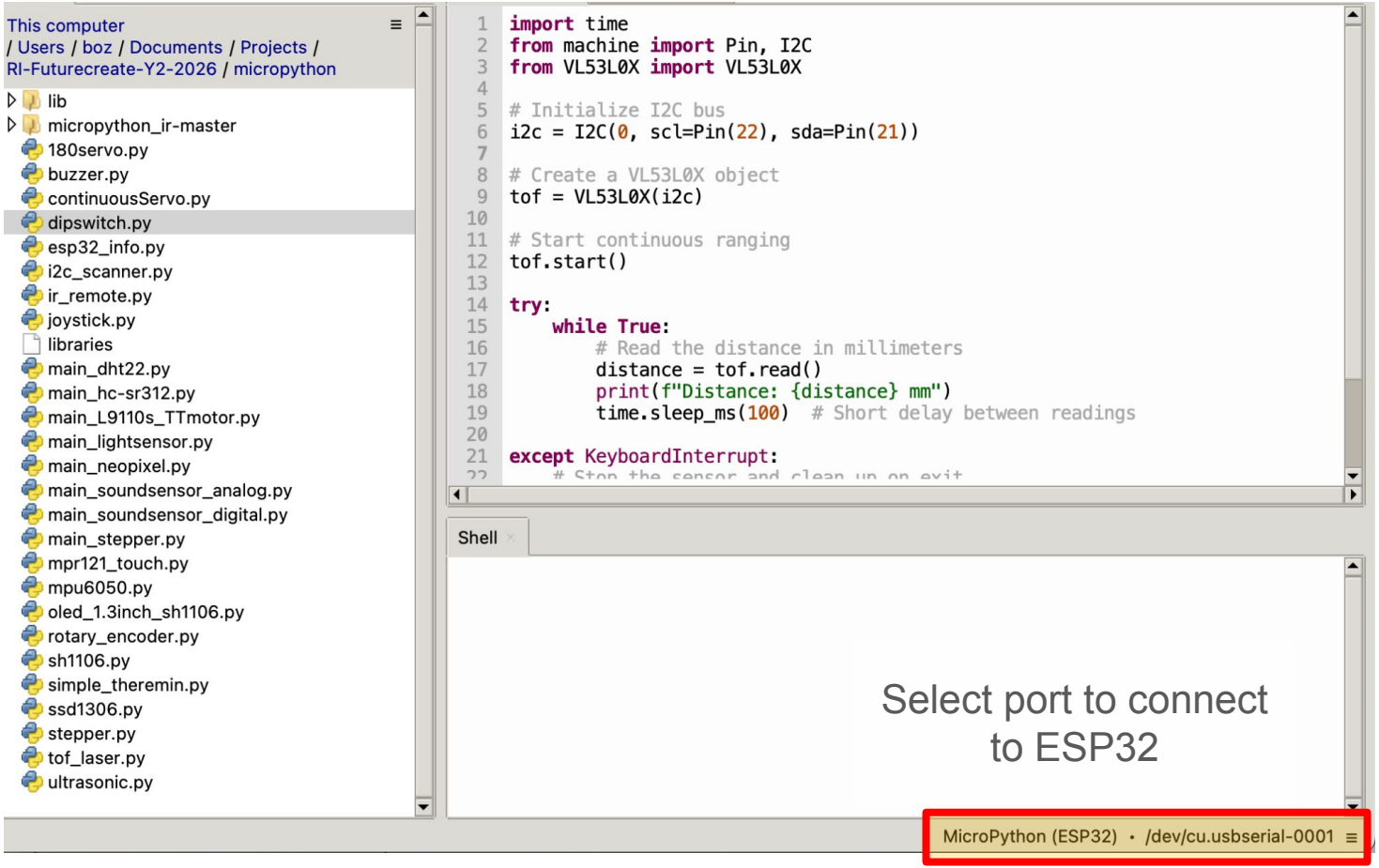
Intro to Thonny IDE



Intro to Thonny IDE



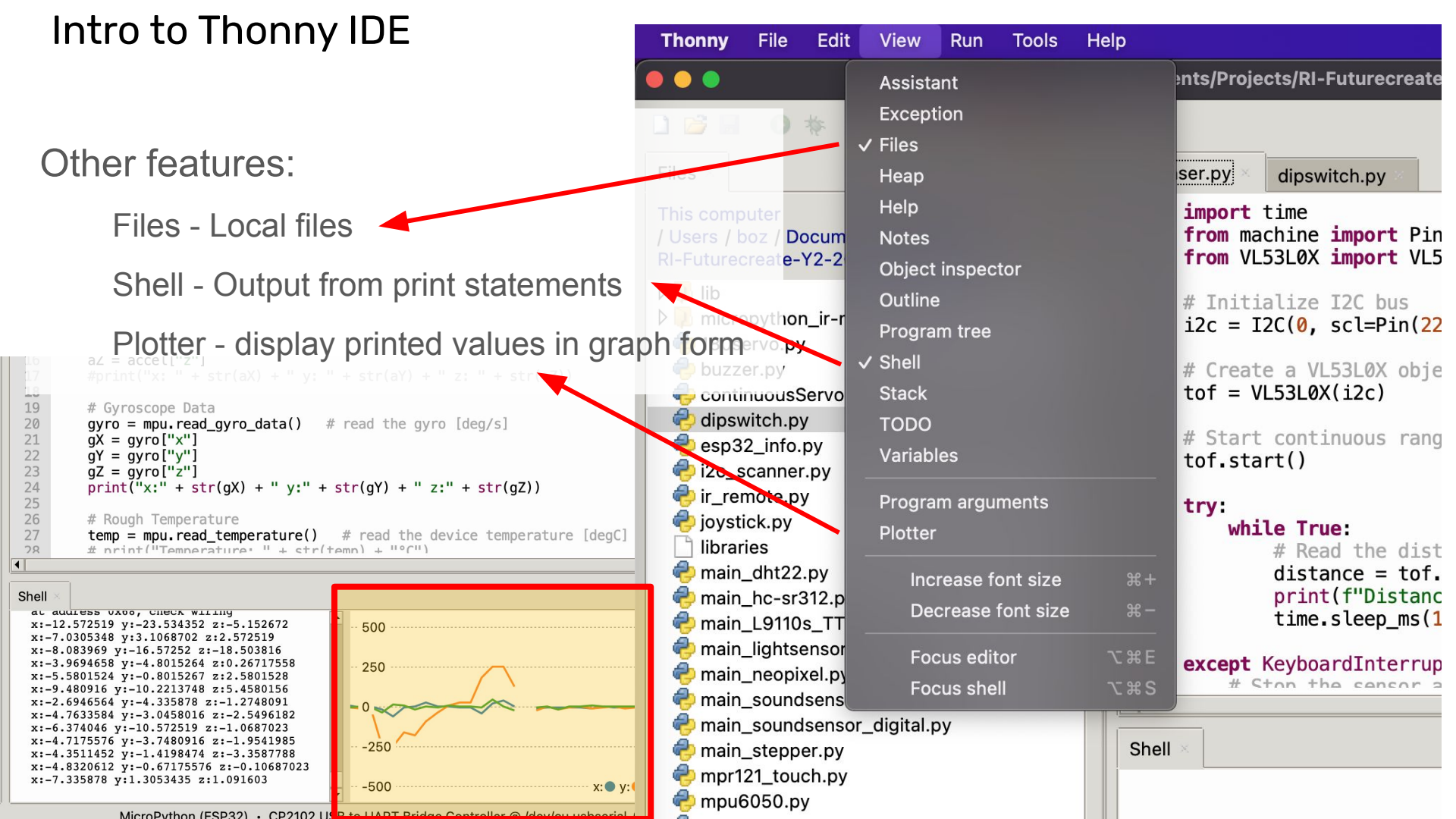
Intro to Thonny IDE



Intro to Thonny IDE

Other features:

- Files - Local files
- Shell - Output from print statements
- Plotter - display printed values in graph form

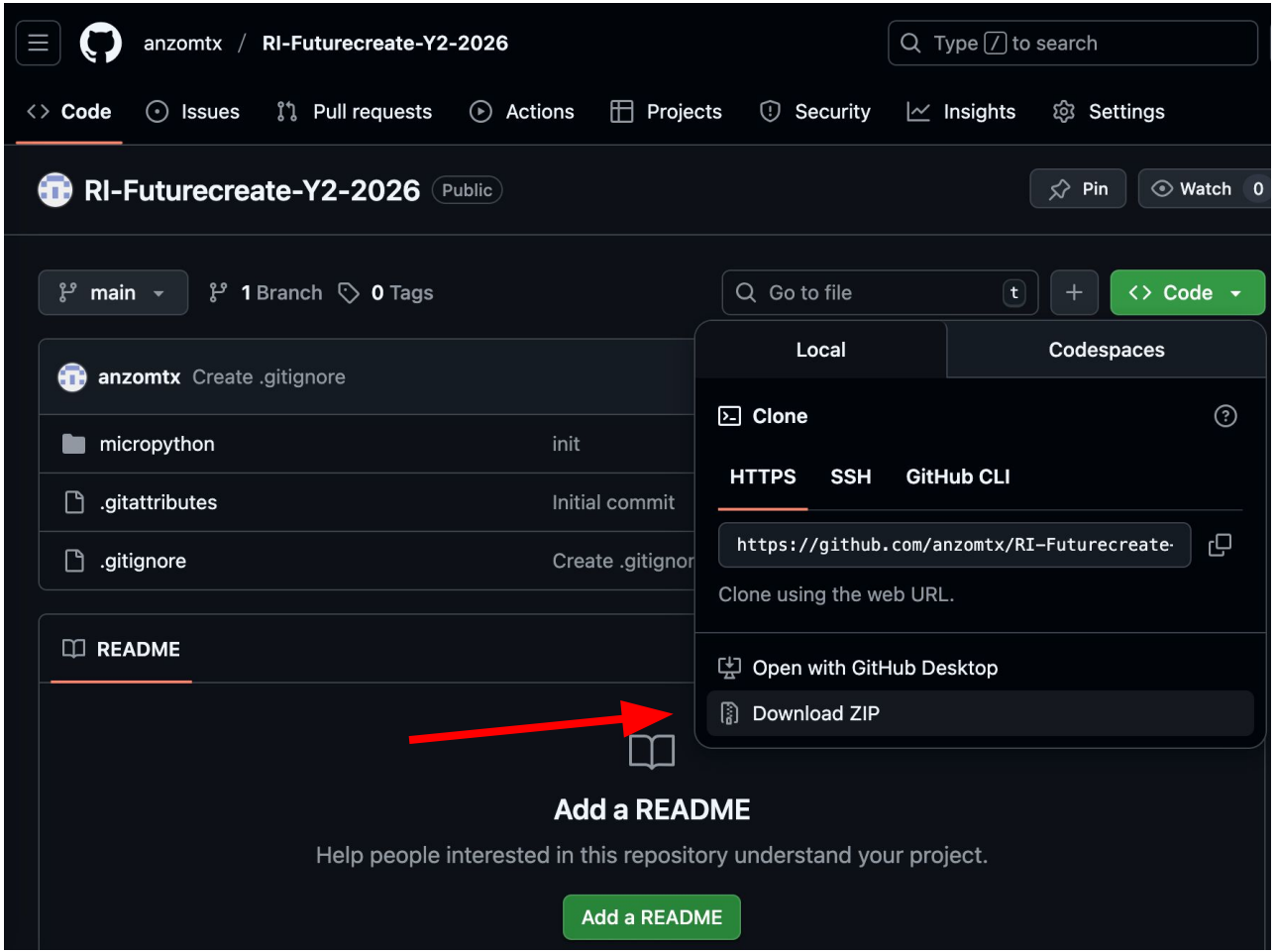


Setup files

Download code:

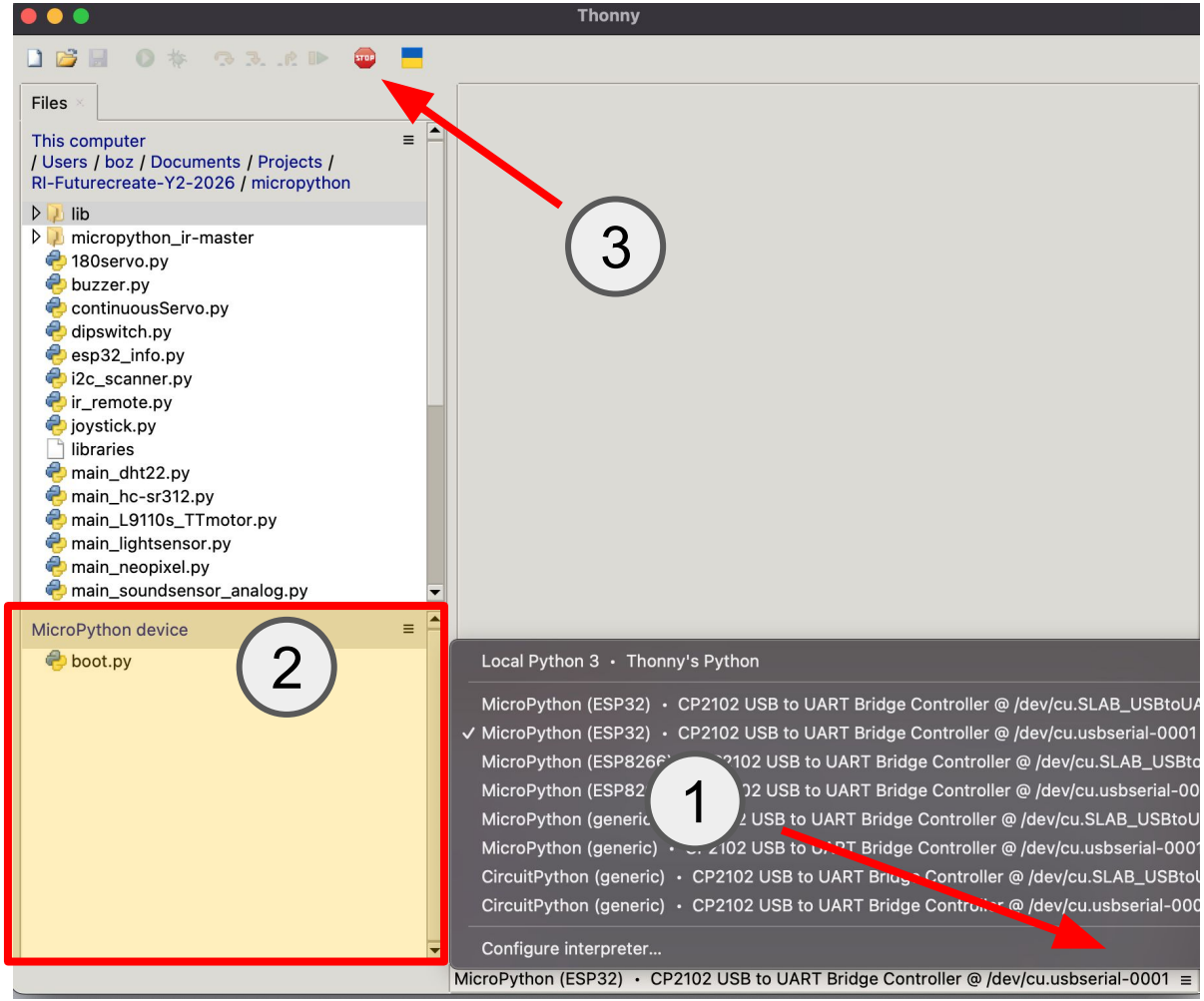
bit.ly/3mis4b

Unzip and move to home folder



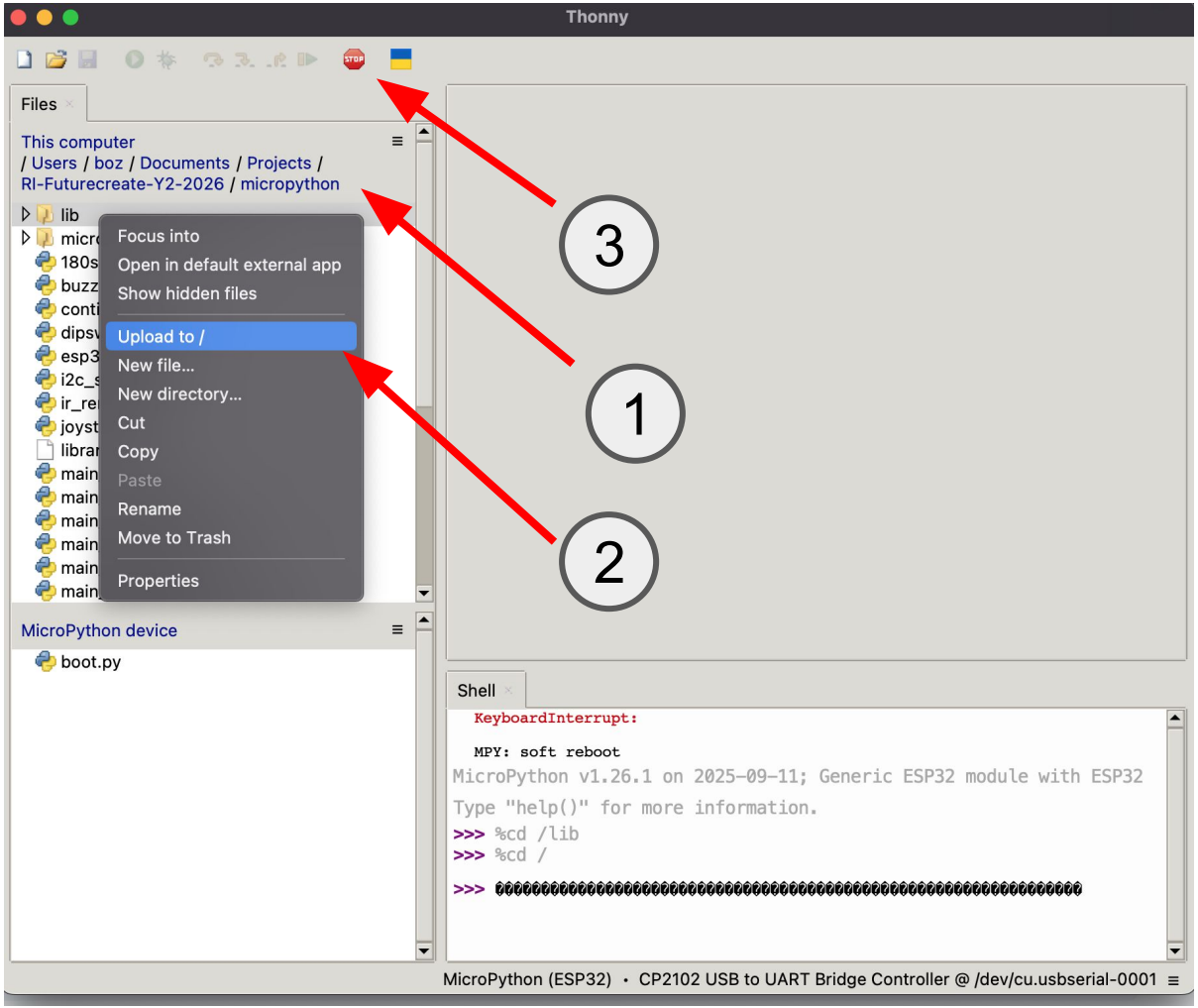
Setup files

1. Select correct port and connect to ESP32
2. If ESP32 is connected, Micropython device's filesystem will appear.
3. You might need to press restart button to reset ESP32



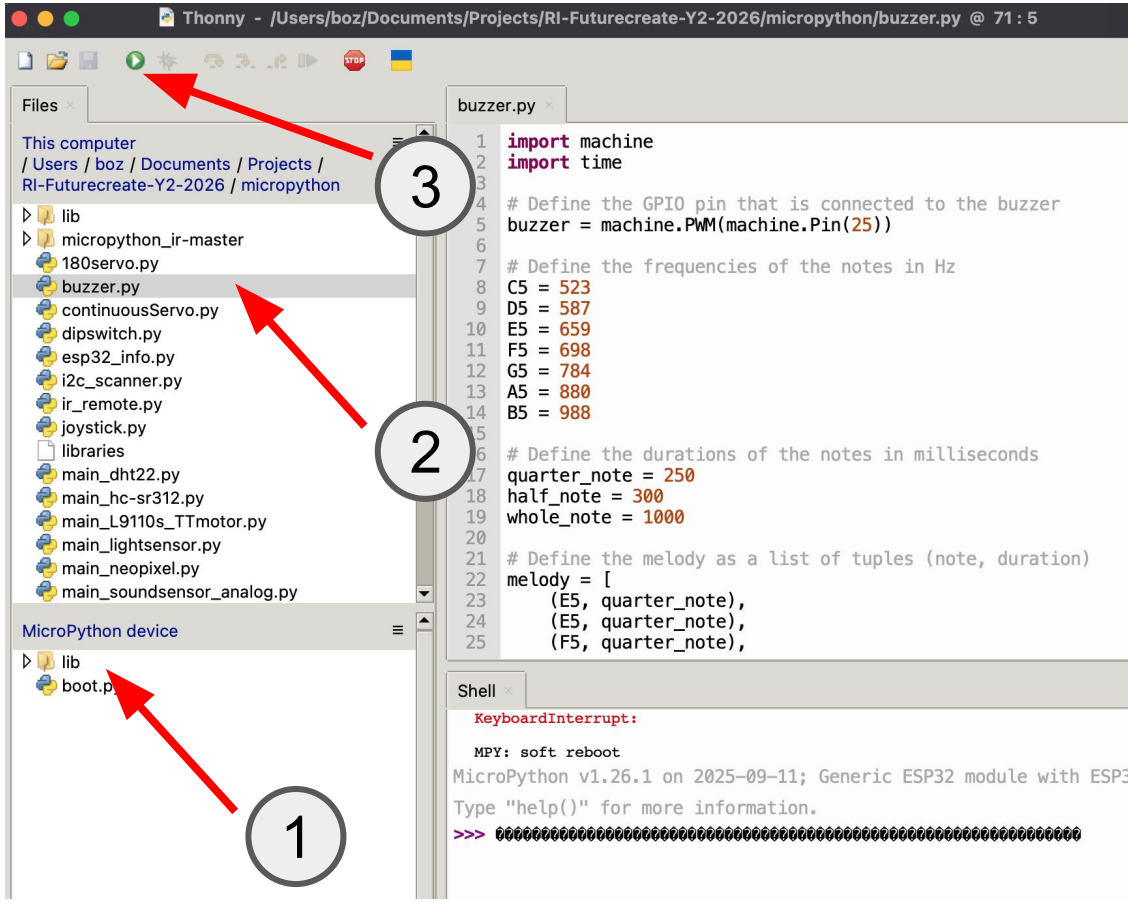
Setup files

1. Ensure local working directory is showing your downloaded code
2. Right-click on ‘lib’ folder and choose **Upload to /**
3. If you encounter errors, press restart button and try again.

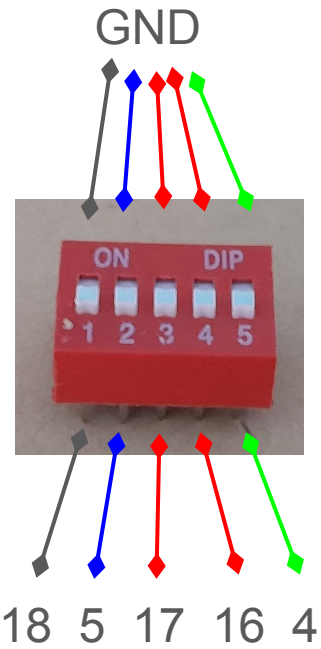


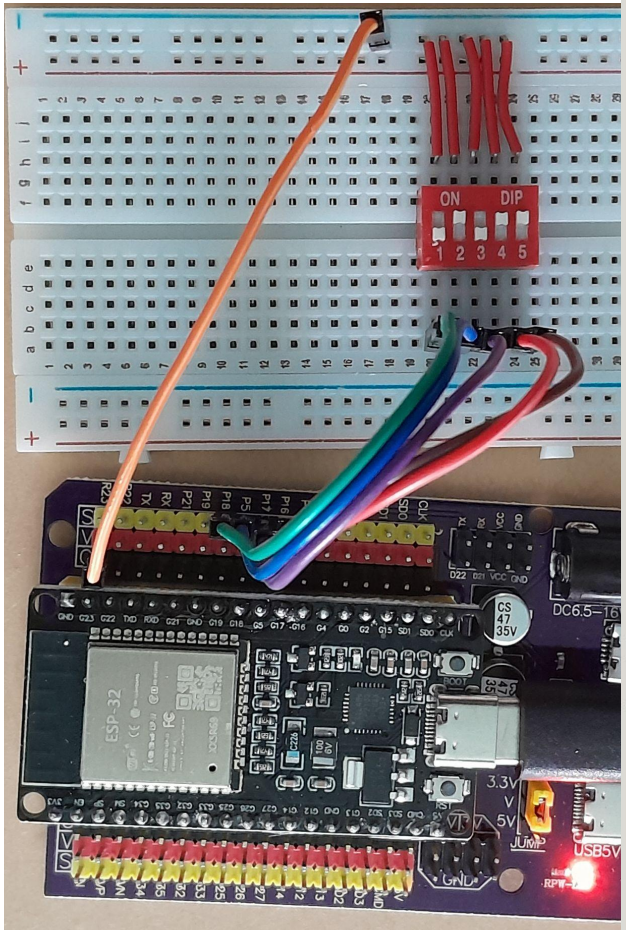
Setup files

1. You should see a ‘lib’ folder on your ESP32 device’s filesystem
2. To open sample code, double click on a file on your local directory
3. Click on run button to execute the code on your ESP32.



Dip switch (switch.py)





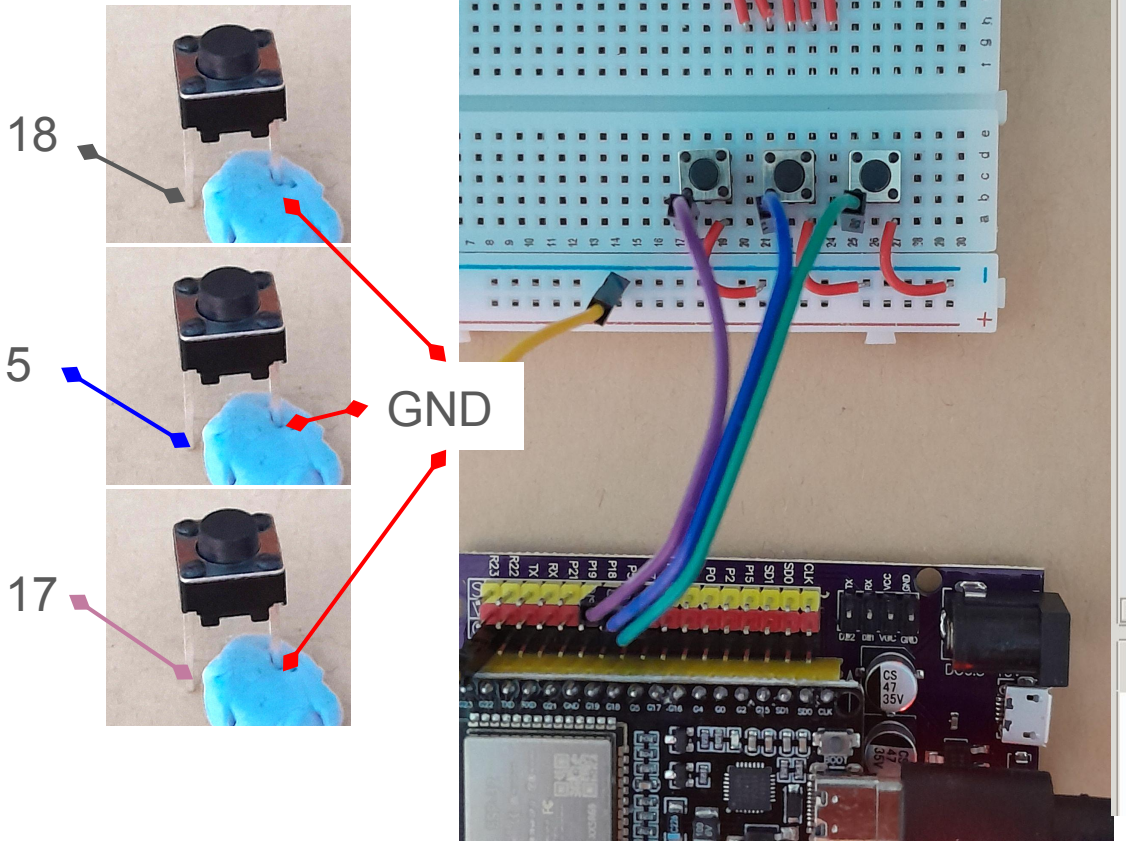
switch.py

```
1  from machine import Pin
2  import time
3
4  # Define the number of positions and the GPIO pins for ea
5
6  SWITCH_PINS = [18, 5, 17, 16, 4]  # Update these based or
7  POSITION_NUM = len(SWITCH_PINS)
8
9  ON = 0
10 OFF = 1
11
12 switches = []
13 for pin_num in SWITCH_PINS:
14     switches.append(Pin(pin_num, Pin.IN, Pin.PULL_UP))
15
16 print("DIP Switch Reader Started...")
17
18 try:
19     while True:
20         # Read and print the state of each switch positio
21         for i in range(POSITION_NUM):
22             state = switches[i].value()
23             if state == ON :
```

Shell

```
Position 3: OFF
Position 4: ON
Position 5: ON

Position 1: OFF
Position 2: ON
Position 3: OFF
Position 4: ON
Position 5: ON
```

Push buttons (switch.py)



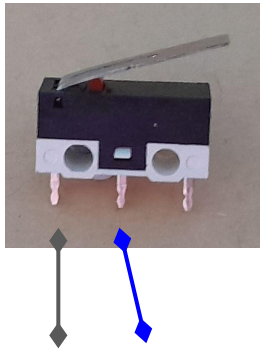
switch.py

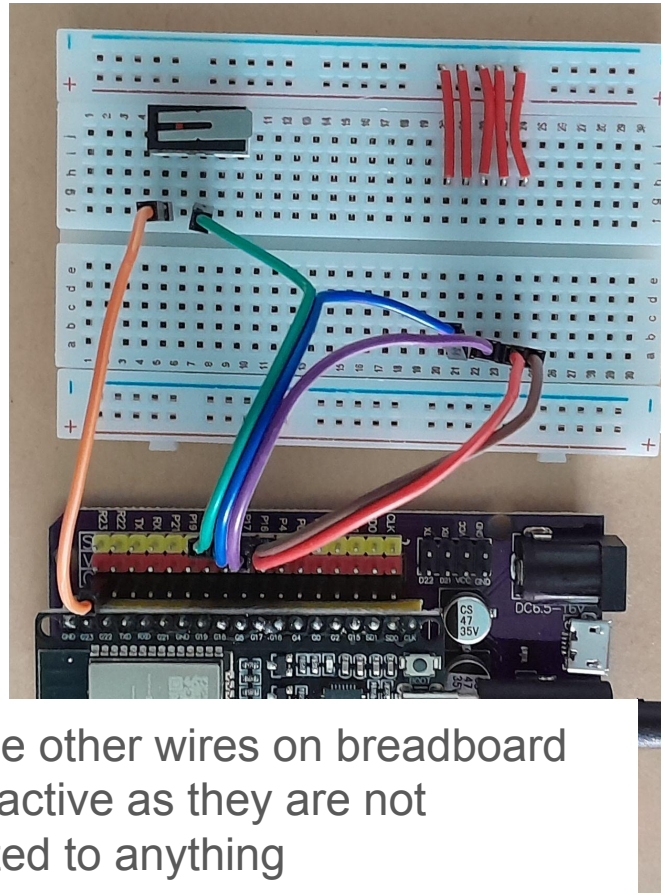
```
from machine import Pin
import time

# Define the number of positions and the GPIO pi

SWITCH_PINS = [18, 5, 17]  # Update these based
POSITION_NUM = len(SWITCH_PINS)

ON = 0
OFF = 1

switches = []
for pin_num in SWITCH_PINS:
    switches.append(Pin(pin_num, Pin.IN, Pin.PUL

print("DIP Switch Reader Started...")

try:
    while True:
        # Read and print the state of each switc
        for i in range(POSITION_NUM):
            state = switches[i].value()
```

Shell

```
Position 3: OFF

Position 1: OFF
Position 2: OFF
Position 3: OFF
```

micro switch (switch.py)





Note: the other wires on breadboard are not active as they are not connected to anything

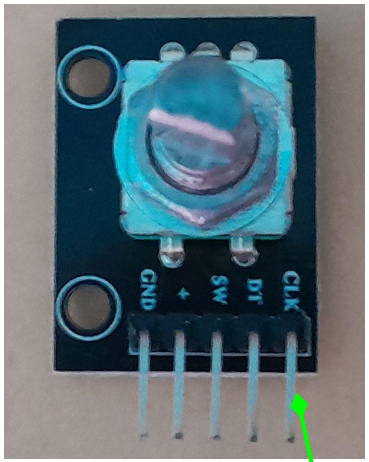
switch.py

```
1  from machine import Pin
2  import time
3
4  # Define the number of positions and the GPIO pins for e
5
6  SWITCH_PINS = [18]  # Update these based on your wiring
7  POSITION_NUM = len(SWITCH_PINS)
8
9  ON = 0
10 OFF = 1
11
12 switches = []
13 for pin_num in SWITCH_PINS:
14     switches.append(Pin(pin_num, Pin.IN, Pin.PULL_UP))
15
16 print("DIP Switch Reader Started...")
17
18 try:
19     while True:
20         # Read and print the state of each switch positi
21         for i in range(POSITION_NUM):
22             state = switches[i].value()
```

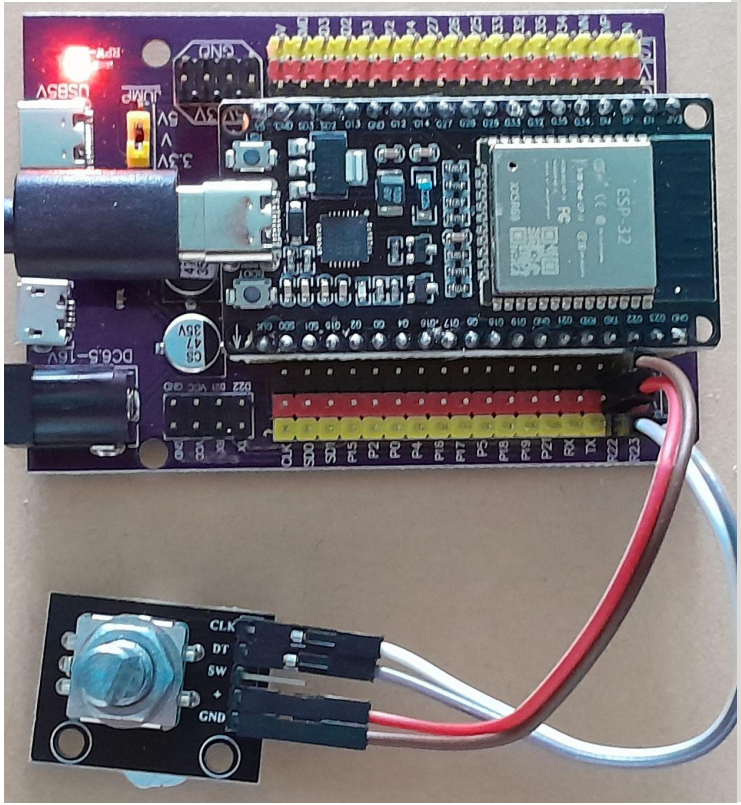
Shell

```
Position 1: ON

Position 1: ON
```

Rotary encoder (rotary_encoder.py)



GND + 22 23



rotary_encoder.py

```
1  from machine import Pin
2  from rotary_irq_esp import RotaryIRQ
3
4  # Initialize the rotary encoder with internal pull-
5  rotary_encoder = RotaryIRQ(
6      pin_num_clk=22,   # connect to CLK
7      pin_num_dt=23,    # connect to DT
8      pull_up=True      # Enable internal pull-up res
9  )
10
11 # Optional: set boundaries and wrapping
12 rotary_encoder.set(min_val=0, max_val=20, range_mod
13
14 last_value = rotary_encoder.value()
15
16 while True:
17     current_value = rotary_encoder.value()
18
19     if current_value != last_value:
20         print("Counter value:", current_value)
21
22         if current_value > last_value:
```

Shell

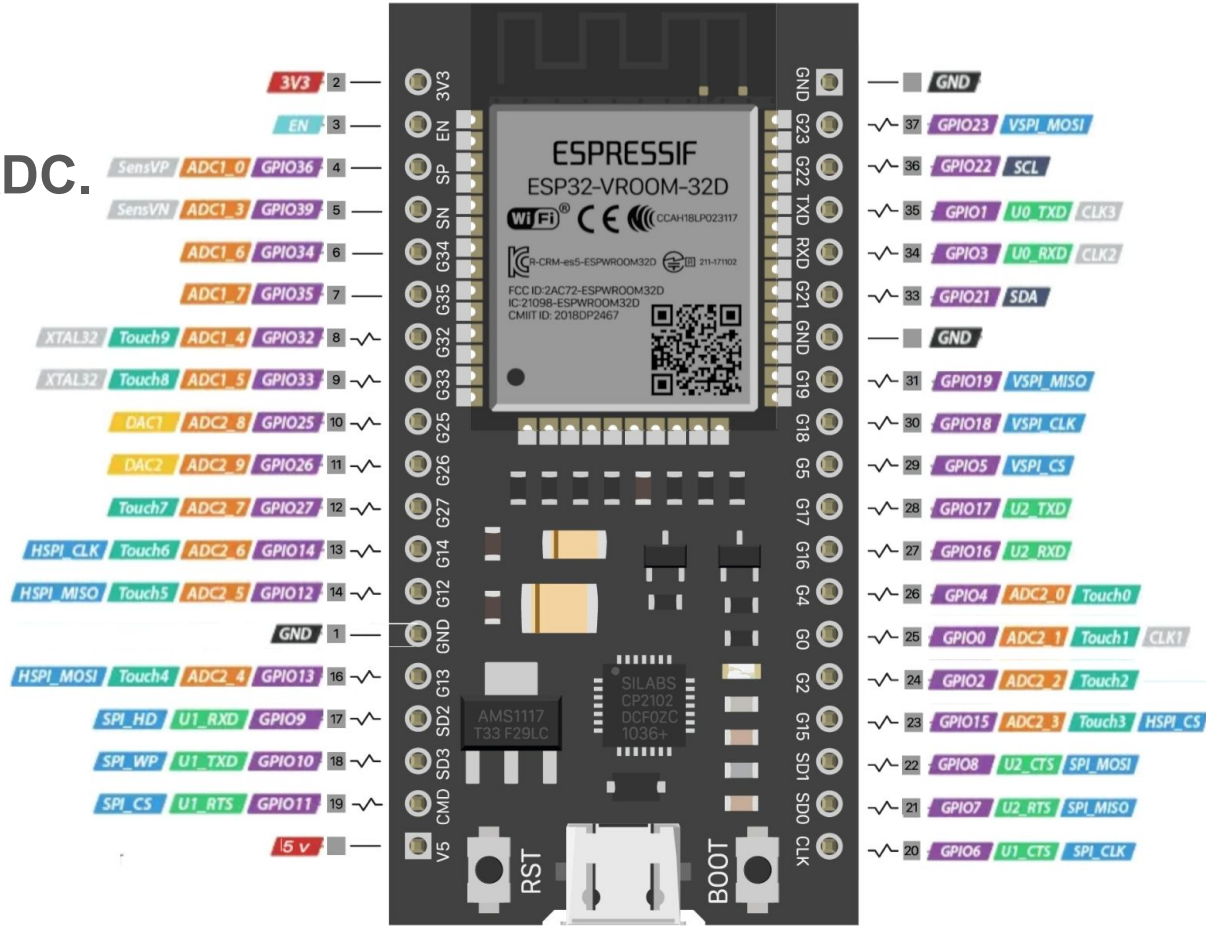
```
Counter value: 1
Direction: decreased
Counter value: 0
Direction: decreased
Counter value: 20
Direction: increased
Counter value: 19
Direction: decreased
Counter value: 18
Direction: decreased
```

Working with ESP32

Only select pins support ADC.

ADC pins:

- 2, 4
- 12, 13, 14
- 25, 26, 27
- 32, 33, 34, 35

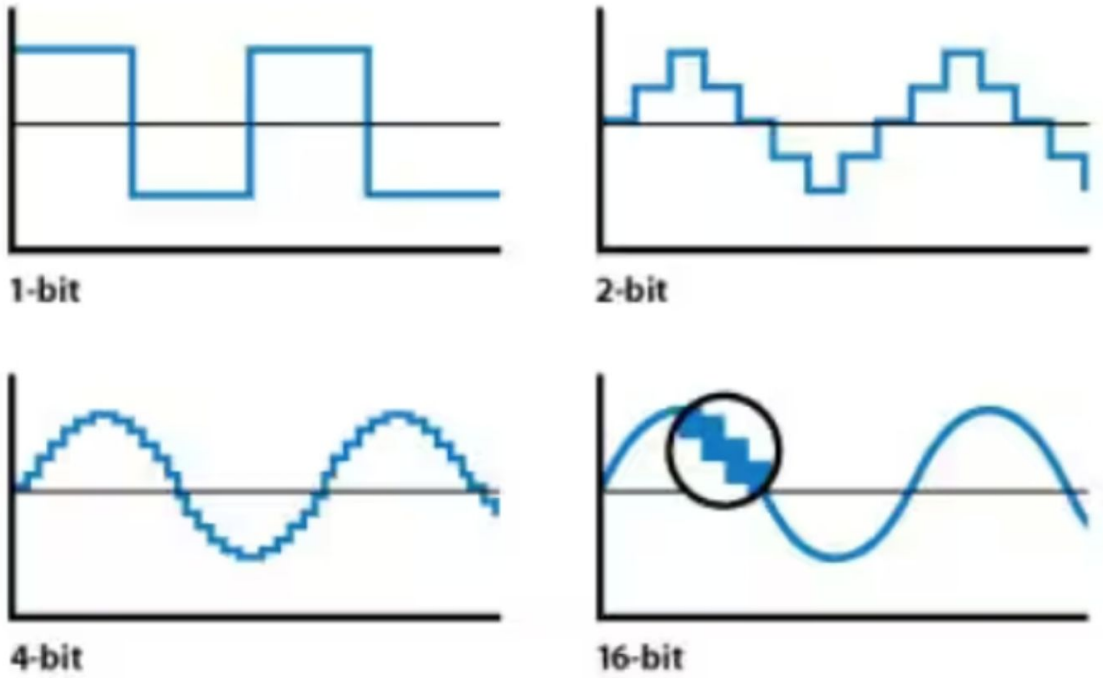


ADC (Analog to Digital) Pins

Digital world of computers only understand binary numbers (0s and 1s)

An ADC pin converts analog signals (like sound, light, temperature, or pressure) into digital signals

‘Bits’ represent resolution, more is better



Joystick (joystick.py)

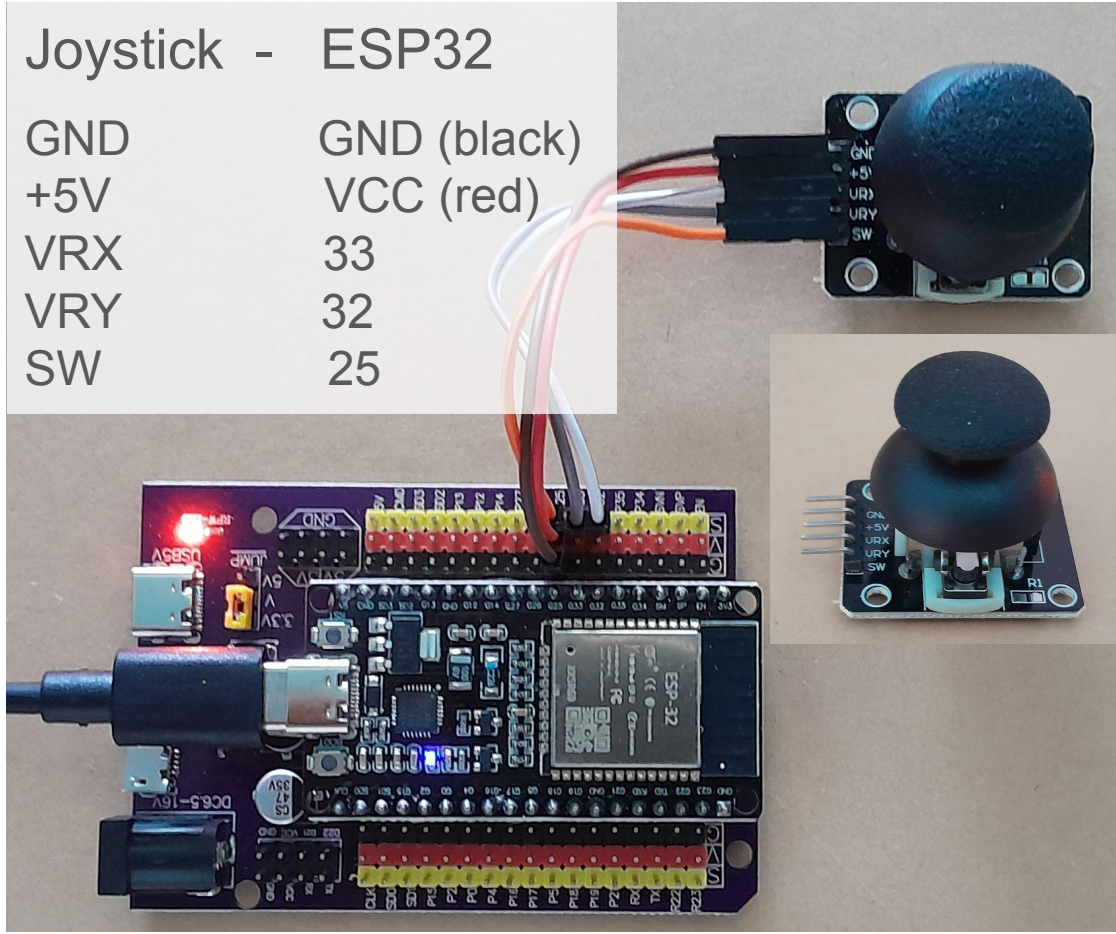
joystick.py

```
from DIYables_MicroPython_Joystick import Joystick
from machine import Pin, ADC
import time

VRX_PIN = 33 # connect to VRX pin of joystick
VRY_PIN = 32 # connect to VRY pin of joystick
SW_PIN = 25  # connect to SW pin of joystick

adc_vrx = ADC(Pin(VRX_PIN))
adc_vry = ADC(Pin(VRY_PIN))
# Set the ADC width (resolution) to 12 bits
adc_vrx.width(ADC.WIDTH_12BIT)
adc_vry.width(ADC.WIDTH_12BIT)
# Set the attenuation to 11 dB, allowing input range up
adc_vrx.atten(ADC.ATTN_11DB)
adc_vry.atten(ADC.ATTN_11DB)

joystick = Joystick(pin_x=VRX_PIN, pin_y=VRY_PIN, pin_
# Configure the debounce time if necessary (default is
joystick.set_debounce_time(100)  # debounce time set to
```

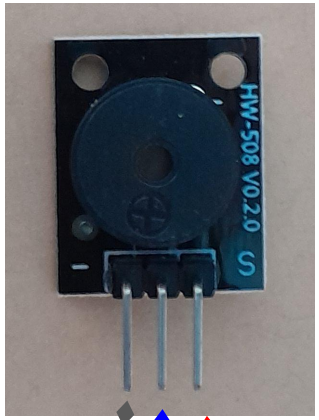
Shell

```
Joystick Position - X: 0, Y: 1927, pressed count: 4
Joystick Position - X: 0, Y: 1945, pressed count: 4
Joystick Position - X: 0, Y: 1930, pressed count: 4
Joystick Position - X: 0, Y: 1933, pressed count: 4
Joystick Position - X: 0, Y: 1931, pressed count: 4
Joystick Position - X: 0, Y: 1931, pressed count: 4
Joystick Position - X: 0, Y: 1931, pressed count: 4
Joystick Position - X: 0, Y: 1930, pressed count: 4
Joystick Position - X: 0, Y: 1922, pressed count: 4
Joystick Position - X: 0, Y: 1945, pressed count: 4
```

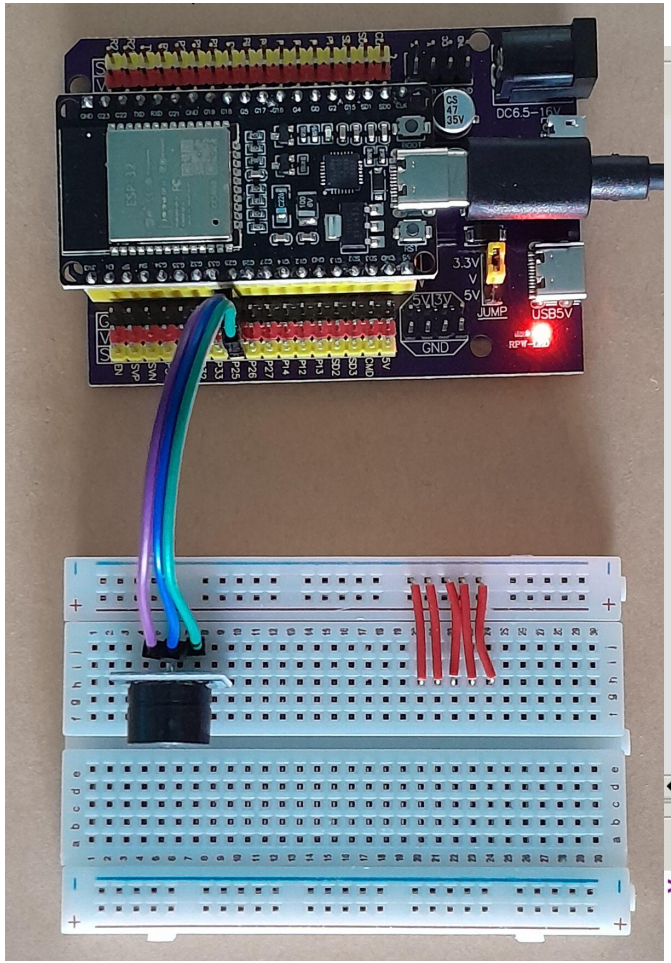
Joystick	-	ESP32
GND		GND (black)
+5V		VCC (red)
VRX		33
VRY		32
SW		25



Making simple sounds with the buzzer (buzzer.py)



GND + 25



buzzer.py

```
1  import machine
2  import time
3
4  # Define the GPIO pin that is connected to the buzzer
5  buzzer = machine.PWM(machine.Pin(25))
6
7  # Define the frequencies of the notes in Hz
8  C5 = 523
9  D5 = 587
10 E5 = 659
11 F5 = 698
12 G5 = 784
13 A5 = 880
14 B5 = 988
15
16 # Define the durations of the notes in milliseconds
17 quarter_note = 250
18 half_note = 300
19 whole_note = 1000
20
21 # Define the melody as a list of tuples (note, duratio
22 melody = [
```

Shell

```
>>> %Run -c $EDITOR_CONTENT
```

Session 1-3 prompt

Design something that turns everyday moments into a show.

If you are dry on ideas…

- A wacky music instrument control panel
- A control panel that works with human movement
- A control panel that looks or acts like a creature